

在 Cloud Run 中使用修訂版本進行流量拆分、漸進式推出及復原

來源：<https://codelabs.developers.google.com/codelabs/cloud-run/revisions-cloud-run-traffic-splitting-gradual-rollout-rollbacks?hl=zh-tw>

步驟數：7

瞭解如何使用修訂版本為 Cloud Run 服務進行流量拆分、漸進式推出和復原作業。

目錄

1. [1. 簡介](#)
2. [2. 設定和需求](#)
3. [3. 流量分配](#)
4. [4. 階段推出](#)
5. [5. 復原](#)
6. [6. 恭喜！](#)
7. [7. 清除所用資源](#)

步驟 1 · 約 0 分鐘

1. 簡介

總覽

您可以透過 Cloud Run 修訂版本指定應接收流量的修訂版本，以及傳送至各修訂版本的流量百分比。您可以透過修訂版本還原至先前的版本、漸進式推出修訂版本，並在多個修訂版本之間拆分流量。

本程式碼研究室說明如何使用修訂版本管理 Cloud Run 服務的流量。如要進一步瞭解修訂版本，請參閱 [Cloud Run 說明文件](#)。

課程內容

- 如何在 Cloud Run 服務的兩個或多個修訂版本之間拆分流量
 - 如何逐步推出新修訂版本
 - 如何復原為先前的修訂版本
-

2. 設定和需求

必要條件

- 您已登入 Cloud Console。
- 您先前已部署 Cloud Run 服務。舉例來說，您可以按照[部署 Cloud Run 服務](#)的說明開始操作。

設定環境變數

您可以設定本程式碼實驗室全程都會使用的環境變數。

```
PROJECT_ID=YOUR-PROJECT-ID
REGION=YOUR_REGION

BG_COLOR=darkseagreen
SERVICE_NAME=traffic-revisions-color
AR_REPO=traffic-revisions-color-repo
```

為服務建立 Artifact Registry 存放區



```
gcloud artifacts repositories create $AR_REPO \
  --repository-format=docker \
  --location=$REGION \
  --description="codelab for finetuning using CR jobs" \
  --project=$PROJECT_ID
```

步驟 3 · 約 0 分鐘

3. 流量分配

本範例說明如何建立 Cloud Run 服務，讀取顏色環境變數，並使用該背景顏色傳回修訂版本名稱。

雖然本程式碼研究室使用 Python，但您可以選擇任何執行階段。

設定環境變數

您可以設定本程式碼實驗室全程都會使用的環境變數。



```
REGION=<YOUR_REGION>
PROJECT_ID=<YOUR-PROJECT-ID>
BG_COLOR=darkseagreen
SERVICE_NAME=traffic-revisions-color
AR_REPO=traffic-revisions-color-repo
```

建立服務

首先，請建立原始碼的目錄，然後 cd 到該目錄。

```
mkdir traffic-revisions-codelab && cd $_
```

接著，請建立 `main.py` 檔案，並加入以下內容：

```

import os
from flask import Flask, render_template_string

app = Flask(__name__)

TEMPLATE = """
<!doctype html>
<html lang="en">
<head>
    <title>Cloud Run Traffic Revisions</title>
    <style>
        body {
            display: flex;
            justify-content: center;
            align-items: center;
            min-height: 50vh;
            background-color: {{ bg_color }}; /* Set by environment variable */
            font-family: sans-serif;
        }
        .content {
            background-color: rgba(255, 255, 255, 0.8); /* Semi-transparent white
background */
            padding: 2em;
            border-radius: 8px;
            text-align: center;
            box-shadow: 0 4px 8px rgba(0,0,0,0.1);
        }
    </style>
</head>
<body>
    <div class="content">
        <p>background color: <strong>{{ color_name }}</strong></p>
    </div>
</body>
</html>
"""

@app.route('/')
def main():
    """Serves the main page with a background color from the ENV."""
    # Get the color from the 'BG_COLOR' environment variable.
    # Default to 'white' if the variable is not set.
    color = os.environ.get('BG_COLOR', 'white').lower()

    return render_template_string(TEMPLATE, bg_color=color, color_name=color)

if __name__ == '__main__':

```

```
port = int(os.environ.get('PORT', 8080))
app.run(debug=True, host='0.0.0.0', port=port)
```

接著，建立 `requirements.txt` 檔案，並加入以下內容：

```
Flask>=2.0.0
gunicorn>=20.0.0
```

最後建立 `Dockerfile`

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8080

ENV PYTHONPATH /app

CMD ["gunicorn", "--bind", "0.0.0.0:8080", "main:app"]
```

使用 Cloud Build，透過 Buildpacks 在 Artifact Registry 中建立映像檔：

```
gcloud builds submit \
  --tag $REGION-docker.pkg.dev/$PROJECT_ID/$AR_REPO/$SERVICE_NAME
```

將第一個修訂版本部署至 Cloud Run，並使用 darkseagreen 顏色：

```
gcloud run deploy $SERVICE_NAME \
  --image $REGION-docker.pkg.dev/$PROJECT_ID/$AR_REPO/$SERVICE_NAME:latest \
  --region $REGION \
  --allow-unauthenticated \
  --set-env-vars BG_COLOR=darkseagreen
```

如要測試服務，可以直接在網路瀏覽器中開啟端點，查看深海綠色的背景顏色。

現在部署第二個修訂版本，並將背景顏色設為棕褐色。

```
# update the env var
BG_COLOR=tan

gcloud run deploy $SERVICE_NAME \
  --image $REGION-docker.pkg.dev/$PROJECT_ID/$AR_REPO/$SERVICE_NAME:latest \
  --region $REGION \
  --set-env-vars BG_COLOR=tan
```

現在重新整理網站時，您會看到背景顏色變成棕褐色。

將流量平均分配

如要在深海綠和棕褐色修訂版本之間分配流量，您需要找出基礎 Cloud Run 服務的修訂版本 ID。執行下列指令即可查看修訂版本 ID：

```
gcloud run revisions list --service $SERVICE_NAME \
  --region $REGION --format 'value(REVISION)'
```

畫面上會顯示類似下方的結果

```
traffic-revisions-color-00003-qoq
traffic-revisions-color-00002-zag
```

執行下列指令並提供修訂版本，即可將流量平分給兩個修訂版本：

```
gcloud run services update-traffic $SERVICE_NAME \
  --region $REGION \
  --to-revisions YOUR_REVISION_1=50,YOUR_REVISION_2=50
```

測試流量拆分

在瀏覽器中重新整理頁面，即可測試服務。

您應該會看到深海綠色修訂版本，另一半則會看到棕褐色修訂版本。輸出內容中也會列出修訂版本名稱，例如：

```
<html><body style="background-color:tan;"><div><p>Hello traffic-revisions-color-
00003-qoq</p></div></body></html>
```

步驟 4 · 約 0 分鐘

4. 階段推出

本節將說明如何逐步將變更推出至新的 Cloud Service 修訂版本。如要进一步瞭解逐步推出功能，請參閱[說明文件](#)。

您會使用與前一節相同的程式碼，但會將其部署為新的 Cloud Run 服務。

首先，將背景顏色設為 `beige`，然後部署名為 `gradual-rollouts-colors` 的函式。

如要將 Cloud Run 函式直接部署至 Cloud Run，請執行下列指令：

```
# update the env var
BG_COLOR=beige

gcloud beta run deploy gradual-rollouts-colors \
  --image $REGION-docker.pkg.dev/$PROJECT_ID/$AR_REPO/$SERVICE_NAME:latest \
  --region $REGION \
  --allow-unauthenticated \
  --update-env-vars BG_COLOR=$BG_COLOR
```

假設我們現在要逐步推出新修訂版本，並將背景顏色設為薰衣草色。

首先，請將目前的修訂版本「米色」設為接收 100% 流量。這樣一來，日後的修訂版本就不會獲得任何流量。根據預設，Cloud Run 會將 100% 的流量傳送至標有 `latest` 旗標的修訂版本。手動指定目前的米色修訂版本應接收所有流量後，帶有 `latest` 標記的修訂版本就不會再接收 100% 的流量。詳情請參閱[說明文件](#)。

```
# get the revision name

BEIGE_REVISION=$(gcloud run revisions list --service gradual-rollouts-colors \
  --region $REGION --format 'value(REVISION)')

# now set 100% traffic to that revision

gcloud run services update-traffic gradual-rollouts-colors \
  --to-revisions=$BEIGE_REVISION=100 \
  --region $REGION
```

您會看到類似 `Traffic: 100% gradual-rollouts-colors-00001-yox` 的輸出內容

現在您可以部署不會接收任何流量的新修訂版本。您不必變更任何程式碼，只要更新這個修訂版本的 `BG_COLOR` 環境變數即可。

如要將 Cloud Run 函式直接部署至 Cloud Run，請執行下列指令：

```
# update color

BG_COLOR=lavender

# deploy the function that will not receive any traffic
gcloud beta run deploy gradual-rollouts-colors \
    --image $REGION-docker.pkg.dev/$PROJECT_ID/$AR_REPO/$SERVICE_NAME:latest \
    --region $REGION \
    --allow-unauthenticated \
    --update-env-vars BG_COLOR=$BG_COLOR
```

現在您在瀏覽器中造訪該網站時，會看到米色，即使薰衣草色是最近部署的修訂版本也一樣。

測試提供 0% 流量的修訂版本

假設您已確認修訂版本部署成功，且放送的流量為 0%。即使通過健康狀態檢查，您仍想確認這個修訂版本是否使用薰衣草色背景。

如要測試薰衣草修訂版本，可以[將標記套用至該修訂版本](#)。標記可讓您在特定網址直接測試新修訂版本，而不提供流量。

首先，請取得最新修訂版本 (薰衣草色) 的圖片網址。

```
IMAGE_URL_LAVENDER=$(gcloud run services describe gradual-rollouts-colors --region
$REGION --format 'value{IMAGE}')
```

現在為該圖片加上相關聯的顏色標記。

```
gcloud run deploy gradual-rollouts-colors --image $IMAGE_URL_LAVENDER --no-traffic --
tag $BG_COLOR --region $REGION
```

您會看到類似下方的輸出內容：

```
The revision can be reached directly at https://lavender---gradual-rollouts-colors-
<hash>--<region>.a.run.app
```

現在只要前往該特定修訂版本的網址，就會看到薰衣草色。

逐步增加流量

現在，您可以開始將流量傳送至薰衣草修訂版本。以下範例說明如何將 1% 的流量傳送至薰衣草。

```
gcloud run services update-traffic gradual-rollouts-colors --region $REGION --to-tags
lavender=1
```

如要將 50% 的流量傳送至薰衣草色，可以使用相同指令，但指定 50%。

```
gcloud run services update-traffic gradual-rollouts-colors --region $REGION --to-tags  
lavender=50
```

畫面上應會顯示各修訂版本接收的流量。

```
Traffic:  
  50% gradual-rollouts-colors-00001-hos  
  50% gradual-rollouts-colors-00004-mum  
    lavender: https://lavender---gradual-rollouts-colors-<hash>-  
<region>.a.run.app
```

準備好全面推出薰衣草色時，您可以將薰衣草色設為 100%，取代米色。

```
gcloud run services update-traffic gradual-rollouts-colors --region $REGION --to-tags  
lavender=100
```

現在造訪網站時，只會看到薰衣草色。

步驟 5 · 約 0 分鐘

5. 復原

假設您收到早期使用者體驗意見回饋，指出顧客偏好米色而非薰衣草色，因此您需要還原為米色。

您可以執行下列指令，復原到先前的修訂版本 (米色)：

```
gcloud run services update-traffic gradual-rollouts-colors --region $REGION --to-revisions $BEIGE_REVISION=100
```

現在造訪該網站時，背景顏色就會顯示為米色。

詳情請參閱[說明文件](#)。

步驟 6 · 約 0 分鐘

6. 恭喜！

恭喜您完成本程式碼研究室！

建議您參閱[推出、回溯及流量遷移](#)相關說明文件

涵蓋內容

- 如何在 Cloud Run 服務的兩個或多個修訂版本之間拆分流量
 - 如何逐步推出新修訂版本
 - 如何復原為先前的修訂版本
-

7. 清除所用資源

為避免產生非預期費用 (例如，如果這個 Cloud Run 函式遭非預期呼叫的次數，超過[免費層級每月 Cloud Run 呼叫次數配額](#))，您可以刪除 Cloud Run 服務，或刪除您在步驟 2 中建立的專案。

如要刪除 Cloud Run 服務，請前往 Cloud Console 中的 Cloud Run (<https://console.cloud.google.com/run/>)，然後刪除您在本程式碼研究室中建立的函式。

如要刪除整個專案，請前往 <https://console.cloud.google.com/cloud-resource-manager>，選取您在步驟 2 中建立的專案，然後選擇「刪除」。刪除專案後，您必須在 Cloud SDK 中變更專案。如要查看所有可用專案的清單，請執行 `gcloud projects list`。
